

The output will be:

```
[sn@localhost ~]$ fg %1
./longprocess1
```

Moving processes to the background or foreground is a valuable technique for effectively managing your terminal sessions in Linux. By using the ‘&’, ‘Ctrl+Z’, ‘bg’, and ‘fg’ commands, you can start, stop, and resume processes as needed, and keep your terminal session organised.

‘jobs’ command

We manipulate background and suspended processes usually via job numbers (JIDs), which, as mentioned earlier, differ from the process ID and are used because they are shorter. The *jobs* command shows you the processes you have suspended and the ones running in the background. For example, if we have three jobs—job1, job2, and job3 and we give the following command at the prompt:

```
$jobs
```

The output will be:

```
[sn@localhost ~]$ jobs
[1] Running                ./job1 &
[2]- Running              ./job2 &
[3]+ Running              ./job3 &
[sn@localhost ~]$
```

The job with the plus sign (+) is the most recent, while the one before that is indicated by a minus sign (-). Linux reassigns job numbers when one completes and another starts.

One of the key benefits of using ‘jobs’ in Linux is that we can manage them as a single entity. For example, you can use the ‘fg’ command to bring a job to the foreground, or the ‘bg’ command to move a job to the background. Understanding the difference between job IDs and process IDs is important for managing terminal sessions in Linux.

Changing process priority in Linux

Each process in Linux is assigned a priority level between -20 and 19, which determines its allotted CPU time. A priority level of -20 represents the highest priority, while 19 represents the lowest priority. By default, Linux assigns most processes a priority level of 0. The ‘nice’ command can start a process at a particular priority level. For example, to start a process named ‘longprocess’ with a priority level of 10, use the following command:

```
$nice -n 10 ./longprocess
```

The ‘renice’ command allows you to change the priority level of a running process. For example, to reduce the priority level of a process with PID of 12034 to 15, use the following command:

```
$renice -n 15 12034
```

The output of the ‘renice’ command is:

```
[sn@localhost ~]$ renice -n 15 20721
20721 (process ID)_old priority 0, new priority 15
```

However, note that though you can use the ‘renice’ command to reduce the priority of the processes, you cannot increase the priority as a non-root user. To increase the priority, run the command with ‘sudo’. For example, to increase the priority of a process with PID 12034 to -10, use the following command:

```
$sudo renice -p -10 12034
```

The output of the ‘renice’ command with ‘sudo’ is:

```
[sn@localhost ~]$ renice -n 5 20721
renice: failed to set priority for 20721 (process ID):
Permission denied
[sn@localhost ~]$ sudo renice -n 5 20721
[sudo] password for sn:
20721 (process ID) old priority 15, new priority 5
[sn@localhost ~]$
```

The ‘-u’ option of the ‘renice’ command allows you to change the priority level of all the processes owned by a specific user simultaneously. For example, to change the priority level of all processes owned by the user ‘ravi’ to 5, type the following at the command prompt:

```
$renice -n 5 -u ravi
```

Assigning higher priority levels to critical processes ensures they receive the necessary CPU time for their tasks. Assigning lower priority levels to non-critical processes prevents them from consuming excessive CPU time and causing other issues. The ‘nice’ and ‘renice’ commands help ensure that critical processes are given the CPU time they require.

The ‘top’ command

The ‘top’ command is a powerful tool for real-time monitoring of processes and their resource usage on a Linux system. When you run this command, it provides an interactive view of the running processes, sorted by the amount of CPU time they are using. The output of the